

TEAMNAME at GroundLM 2026 Shared Tasks: Reading the Scorer — Metric Decomposition and Annotation-Convention Recovery for Literature-Grounded QA

Author Name

Affiliation

email@example.com

Abstract

We describe our submission to LitTraceQA, the literature-grounded question answering task at GroundLM 2026. The task scores three coupled outputs — paper identifiers, coarse evidence locators, and answers in multiple-choice or table form — and our final official score on the 71-question held-out test split is **0.7647**, up from **0.4563** for our first submission and **0.5519** for our best fully automated one. Almost none of that improvement came from the retrieval-and-reading pipeline we started with. It came from two other places. First, we read the released evaluator source and found that the metric is far more asymmetric than it looks: paper F1 and evidence F1 carry weight 1/3 each while each of the three answer metrics carries only 1/9, and the evidence key is a coarse tuple in which a correct value on a correct page still scores zero if the *source type* or the visible object id is wrong. Second, we treated the 55-example development split as a corpus for recovering the *annotation conventions* rather than as a validation set for tuning, and we treated our own leaderboard history as a measuring instrument: because the macro metrics are sums of a small number of rational per-question values, a single controlled change to one prediction file often admits exactly one arithmetic explanation. This let us prove specific predictions right or wrong rather than guess. We report what this recovered (gold uses one evidence key per paper; gold table rows equal gold paper count in 8 of 8 multi-paper development questions; gold row keys reproduce labels printed in the source paper), what it refuted (three separate hedging heuristics, each measured and dropped), and the honest limitation that a substantial share of our final score reflects per-question auditing and leaderboard-feedback attribution rather than a system that would generalise to new questions.

1 Introduction

LitTraceQA asks a system to answer a research question by returning three things at once: the canonical identifiers of the paper or papers involved, coarse evidence locations inside those papers, and the answer itself in a requested format. The interesting property of the task is that these are scored separately, so a system can be right for the wrong reasons in a way the score makes visible — a correct multiple-choice answer with a wrong paper, or a correct paper with a locator pointing at the wrong page.

Our participation had an unusual shape and we want to be direct about it up front. We built a conventional pipeline (dense retrieval over the released metadata pool, cross-encoder reranking, an LLM selector, and a vision-capable reader for tables and figures), it reached an official score of 0.4563, and then it stopped improving. Every subsequent gain — to 0.7647 — came from work that is better described as *instrument reading* than as modelling: studying the scorer, recovering the annotation conventions of the dataset from the development split, and using controlled single-variable submissions to identify which of our predictions were wrong.

We think the resulting findings are more useful to future participants than our architecture is, so this paper is organised around them. Section 2 decomposes the metric. Section 3 describes the pipeline. Section 4 reports the annotation conventions we recovered. Section 5 describes score-guided attribution and its limitations. Sections 6–8 give official results, error analysis, and the heuristics we measured and discarded.

2 The Metric Is Asymmetric

The released evaluator computes

$$\text{answer} = \frac{\text{mc} + \text{row_f1} + \text{cell_acc}}{3}$$

$$\text{overall} = \frac{\text{paper_f1} + \text{evidence_f1} + \text{answer}}{3}$$

so paper F1 and evidence F1 each carry weight 1/3, while multiple-choice accuracy, table row F1 and table cell accuracy carry 1/9 each. Two-thirds of the score is therefore *provenance*, not answering. This is the single most consequential fact about the task and it is easy to miss: an intuition that answer accuracy is the objective leads to effort in the wrong place. On our first submission, multiple-choice accuracy was already 0.68 while evidence F1 was 0.3587.

The evidence metric deserves a second look. Predictions and gold are compared as sets of coarse keys

$(\text{paper_id}, \text{source_type}, \text{page}, \text{object_id})$

where *object_id* is the normalised visible identifier for table, figure, equation_algorithm and citation_context, and is *empty* for text_span. Three consequences shaped our work. (i) A *text_span* item matches on paper and page alone, so it is the cheapest correct guess when the type is uncertain. (ii) Conversely, citing the right page with the wrong *source_type* scores zero — “we found the answer” is not the same as “we located it as the annotator did”. (iii) Because scoring is set-based and macro-averaged per question, emitting N predictions against G gold items with C correct gives $F_1 = 2C/(G + N)$, which makes the cost of hedging exactly computable rather than a matter of taste. We use this repeatedly.

The table metric has a similar structure. Row F1 is a set F1 over tuples of the row-key columns, so duplicate row keys silently collapse in the evaluator’s dictionary (last one wins). Cell accuracy iterates over *gold* rows only: cells belonging to a predicted row whose key does not match gold are never looked at, and unmatched gold rows count their cells as wrong. Cell accuracy is therefore bounded by row *recall* and is completely indifferent to row precision — which means an extra candidate row can cost row F1 but can never cost cell accuracy. This asymmetry is the basis for several of our decisions in Section 4.

3 System

Retrieval and selection. The released pool contains 27,487 papers with title, abstract, venue, and a PDF URL. We embed question and title–abstract

text, retrieve a candidate list, rerank with a cross-encoder, and pass the top candidates as a numbered list of titles and abstracts to an LLM selector that returns indices. Enabling reranking moved paper F1 from 0.410 to 0.490 on a development family, and the LLM selector plus a cap of three papers per question was our best configuration (`--llm-select --visual-table --max-papers 3`).

We measured shortlist recall separately to know whether selection or retrieval was the bottleneck: on a test-like development family, recall of the gold paper was 0.846 at rank 1 and 0.962 at rank 20, and *flat* out to rank 200. Widening the shortlist was therefore pointless, and we did not pursue it.

Reading. Selected PDFs are fetched and read locally with PyMuPDF for page text, caption indices, and page rasterisation. Answer generation is a per-question prompt carrying the question, the requested answer types, the table schema when one is supplied, and page text or rendered page images. Table questions use a two-stage scheme in which row keys are extracted from the question alone before values are read, guarded by a check that every extracted key actually occurs in the question — without that guard the stage fabricated entities and cost 0.129 overall.

Audit loop. The component that produced most of our score is not a model but a procedure. Each candidate submission is checked question by question against the source PDFs: does the cited page contain the claim, does the cited object id exist in that page’s captions, does every emitted paper mention an entity the question names, does the chosen multiple-choice option match the row and column the question actually asks about. We automated the checks that could be automated (Section 6) and performed the adjudication with an LLM agent reading the rendered pages. This is a human-in-the-loop system in the sense that matters for interpreting our score, and we return to it in Section 10.

4 Recovered Annotation Conventions

The development split has 55 examples with gold papers, evidence and answers. Used as a validation set for tuning it is too small to separate configurations. Used as a corpus describing *how the annotators behaved*, it was the most valuable resource in the task. Every statement below is a count over that split; Table 1 collects them with their denomi-

Convention (counted on the 55 dev ques- support tions)	
Exactly one evidence key per gold paper of which (1, 1) / (4, 4) patterns	43/55 24 / 19
Gold table rows = gold paper count (multi-paper)	8/8
Question names a figure $\Rightarrow$ gold has figure	10/10
Question names a table $\Rightarrow$ gold has table	3/3
Question asks an equation/loss $\Rightarrow$ gold has <code>equation_algorithm</code>	4/7
Gold pairs a <code>text_span</code> with a same- page object	1/64
Gold evidence page is the <i>earliest</i> occur- rence of the value	10/38
middle / latest	22 / 6

Table 1: Annotation conventions recovered from the development split, with the counts they rest on. The two strongest (rows 3–4) were the most productive on test; the last two are why we did *not* ship a positional page-rewrite rule or a same-page `text_span` pad.

nators, because several rest on small samples and we would rather a reader see that than take them on trust.

One evidence key per paper. Comparing the number of distinct coarse evidence keys to the number of gold papers gives (1, 1) 24 times, (4, 4) 19 times, plus (3, 3) and (9, 9); 43 of 55 questions have exactly one key per paper. Gold evidence is therefore *provenance per paper*, not an exhaustive list of supporting locations. We sized our evidence sets accordingly.

Source type follows the question’s wording. Conditioning gold’s type on the question’s vocabulary is strikingly predictive. Of the questions whose wording names a figure, plot or chart, **10 of 10** have gold containing `figure`; for wording that names a table, 3 of 3; for wording that asks for an equation, expression, loss or objective, 4 of 7 contain `equation_algorithm`, and 2 of those 4 also carry `text_span`. Acting on this was our single largest evidence gain: we had been emitting *no* `equation_algorithm` items at all. Adding them alongside `text_span` on the six equation-form questions coincided with official evidence F1 moving from 0.6610 to 0.7121; that submission also re-aimed two evidence pages, so the whole delta is not attributable to the type change alone. Where the type is genuinely mixed in gold, carrying both is the right arithmetic: against a single gold item, two predictions with one hit score 2/3, where committing to one scores 0.4 in expectation

at $P(\text{equation}) = 0.6$.

Gold table rows equal gold paper count. For every multi-paper table question in the development split — (4, 4) six times, (3, 3), (9, 9) — gold has exactly one row per paper, 8 of 8. This is a hard structural constraint, and it does more than cap row F1: if gold has two rows, then gold’s two keys cannot be two of our four *quantity descriptors*, because a two-row table cannot key its rows by four separate quantities. Three of our test predictions violated the constraint and were, by this argument, scoring zero.

Row keys reproduce printed labels. Where the question names entities, gold keys are those names lowercased, keeping qualifying parentheticals (`ecm-x1 (102.4m)`). Where the question implies variants of one paper, gold uses the labels *printed in that paper’s own table* — one development question keys its rows w/ `ground truth` and w/ `cube rcnn`. Checking every test row key against its source paper this way found four mismatches, of which the decisive one was a retrieval question whose gold keys carry the paper’s `+Fake2` suffix rather than the bare dataset names we had used. Correcting it took that question’s row F1 from 0 to 1.0 and its cell accuracy from 0 to 1.0 in one step.

Evidence pages are not the first mention. Gold’s page histogram peaks at pages 6–7 (65 of 149 items) with only 5 of 149 on page 1, while our predictions put 40% of their items on pages 1–3. The obvious correction is wrong, though: among gold items whose answer value appears on more than one page, gold is the earliest occurrence 10 times, somewhere in the middle 22 times, and the latest 6 times. There is no positional rule to learn, only a semantic one. We used the statistic as a triage signal for manual review and explicitly did not ship it as a rewrite (Section 8).

5 Score-Guided Attribution

The task permits five evaluator submissions per day. Because each macro metric is a mean of a small number of rational per-question values, a submission that changes one thing often has exactly one arithmetic explanation, and the feedback identifies *which* of our predictions were wrong rather than merely whether the file got better.

Two examples. Table row F1 sat at exactly 0.340476 for five consecutive submissions, so

when two questions’ row keys were changed and the macro moved by $+0.011111$ — that is, $+0.2333$ over the 21-question sum — the shift decomposed uniquely as one question going from zero matched rows to one while its row count went from 2 to 4 ($2 \cdot 1/6 - 0 = +0.3333$) and a second question’s added candidate missing ($2 \cdot 1/5 - 2 \cdot 1/4 = -0.1$). That told us both that gold’s paper row keys are short names rather than full metadata titles, and that one specific candidate string was wrong. Second, `table_cell_accuracy_micro` is `cell_correct/cell_total` and `cell_total` is a property of gold: reading it as an exact fraction gives $23/87$ and later $31/87$, so gold has exactly 87 comparable cells and the $+8$ was one question’s eight cells.

The same instrument disciplined us. A later submission gained $+0.3333$ where we had predicted $+0.5833$; because three of the changed questions had identical key sets before and after, their contributions had to cancel, which forced the conclusion that a change we had reasoned was “free” had in fact discarded a matching key. We reverted it.

We flag clearly that this is adaptation to the held-out split. It is within the stated submission rules, and we report it because it materially explains our final number; but scores obtained this way should not be read as an estimate of how the underlying system would perform on unseen questions.

6 Official Results

Table 2 reports official evaluator output for our scored submissions on `littraceqa-test` (71 questions). All figures are as returned by the evaluator; no numbers in this paper are computed by us from private labels.

Where the score came from. Between our first and last submission, paper F1 rose by 0.338 and evidence F1 by 0.368; multiple-choice accuracy rose by 0.30 but contributes at one third the weight. Table row F1 and cell accuracy remain our weakest components at 0.5151 and 0.2976, and together they account for the majority of our remaining headroom.

Automated checks. Four checks ran over every candidate file and are the reusable part of our tooling. (1) An *object-id verifier* regexes captions out of each page and confirms the cited `table_id/figure_id` exists on the cited page; 48 of 49 of our ids passed, and the one failure was

a real error (a retention-score column we had attributed to Table 2 when its caption sits on the next page). (2) A *name-presence check* asks whether each emitted paper’s text ever mentions an entity the question names — a paper that never writes “ERASE” is not the ERASE paper. (3) The same check against all 27,487 titles, which finds papers that were never retrieved and so could not be checked any other way. (4) A *dead-run guard* that fails a submission containing all-null table rows; this caught two files silently produced after our API quota was exhausted mid-run, with no error in the trace.

7 Error Analysis

The dominant failure is the right paper and the wrong row. Of the answer errors we found and corrected, almost none are hallucinations. They are values genuinely printed in the cited paper, one row or one column away from the one the question asks about: the un-finetuned base-model row instead of the fine-tuned one; `RPEt` where `RPEr` was asked; the `MC1` column where the question says `MC3`; inter-annotator agreement where the question says intra-annotator; a *Pearson* column read as a Kendall one; the *None* baseline row’s accuracy reported as the proposed method’s macro-F1. This is why three separate value-presence triages we built returned mostly false positives — every candidate value *is* in the paper — and why reading row and column headers is the only check that works.

Wrong papers hide behind plausible titles. Seven of our paper sets were wrong and every one was found by the presence checks rather than by re-running selection. Two required fetching a PDF that had never been retrieved, so no text-based check could have reached them, and one was a question answered “all three papers” while citing a paper that was none of the three the options named.

Figures need rasterising. Four questions ask for counts that exist only in a figure. Rendering those pages at $2\times$ and reading them settled all four and corrected one answer: a paper states in prose that it classifies models into *eight* categories, and its pie chart labels agree, where we had answered six.

Grader-invented strings are not recoverable. Where a table’s row-key column is a descriptor the annotator invented (`quantity`, `setting`, `attribute`, `metric`), we never matched it. One such cell was attempted in four different forms

Submission	paper F1	evid. F1	MC	row F1	cell acc	overall
initial pipeline	0.6324	0.3587	0.68	0.3221	0.1310	0.4563
+ LLM selector, visual tables	0.7991	0.4737	0.78	0.2738	0.0952	0.5519
+ hand-authored table row keys	0.7991	0.4737	0.78	0.3405	0.0952	0.5602
+ answer audit vs. PDFs	0.8554	0.5347	0.84	0.3405	0.1984	0.6166
+ audit, continued	0.8789	0.5606	0.86	0.3405	0.2103	0.6366
+ 7 paper-set corrections	0.9704	0.6610	0.94	0.3405	0.2103	0.7095
+ equation_algorithm evidence	0.9704	0.7121	0.96	0.3405	0.2103	0.7287
+ figure-counting answers	0.9704	0.7121	0.98	0.3516	0.2103	0.7322
+ short-name table row keys	0.9704	0.7121	0.98	0.4283	0.2262	0.7425
– rows proven not to match	0.9704	0.7121	0.98	0.4675	0.2262	0.7468
+ batched cell rewrites	0.9704	0.7192	0.98	0.4675	0.2500	0.7518
+ printed-label row keys	0.9704	0.7262	0.98	0.4873	0.2976	0.7617
– rows proven not to match (2)	0.9704	0.7262	0.98	0.5032	0.2976	0.7634
+ three row sets restored (final)	0.9704	0.7262	0.98	0.5151	0.2976	0.7647

Table 2: Official evaluator results on the LitTraceQA held-out test split. Rows are in submission order; each line names the change that distinguishes it from the line above. Paper F1 and evidence F1 carry weight 1/3 each in *overall*; MC, row F1 and cell accuracy carry 1/9 each. Components are the evaluator’s reported values rounded to four decimals, so recomputing *overall* from them can differ in the last digit. Row two is our best fully automated submission; every later row involves per-question intervention.

— the verbatim question clause, a bare noun phrase, the same with a “number of” prefix, and the source paper’s own noun — and all four missed. We report this as a limit of the approach rather than as an unexplored lead.

8 Negative Results

We list heuristics that seemed reasonable, were measured, and were dropped. We think these are the most transferable part of the paper.

- **Padding paper sets.** From one submission’s precision and recall alone, the gold set sizes on test are derivable as 1 or 2 (3 and 4 are arithmetically impossible), and our selector is size-calibrated: an uncalibrated selector cannot reach the observed precision even if its second pick were always gold. Padding singletons was therefore refused.
- **Full-text indexing.** Content-defined multi-paper questions look like they need body text, but only 4% of the relevant development family is answerable from body text alone; the shortlist was already saturated (Section 3).
- **Re-aiming evidence at later pages.** Motivated by the page histogram, measured at +0.013 evidence F1 on development — below our shipping threshold — and explained by the middle-position statistic.
- **Carrying both row-key spellings everywhere.** Measured at −0.020 row F1 on development, because there the bare keys are

usually already right and the alternative never hit. We restricted hedging to keys we could *prove* wrong.

- **Adding a table item wherever a text-span page has a table caption.** Initially recorded as “no effect”; on re-examination the experiment added only 4 items and all 4 landed on questions already scoring zero, so it was *uninformative rather than refuted*. We did not ship it, and we correct the earlier reading here.
- **Retyping text_span to table.** Of 44 wrong evidence items examined, 41 were page-level errors and only 2 were type errors, so a blanket retype had no target.

We also record two errors in our own instrumentation, because both produced conclusions we briefly believed. A multiple-choice triage stripped whitespace before matching numbers, which glues adjacent table cells together, so 16,825,000 5,800,000 became one token and both halves were reported absent; every “value not in this paper” verdict it produced was worthless until whitespace was collapsed rather than removed. And a prediction-file diff compared a key that prediction files do not carry, so both sides read `None` and the diff reported “0 of 71 identical” when the true answer was 26.

9 Setup and Disclosed Resources

Data. Released LitTraceQA metadata pool (27,487 papers), development split (55), and input-only test split (71), from the official Hugging Face

dataset (CC BY-NC 4.0). PDFs were fetched from the publisher URLs in the released metadata. We used no additional annotated data.

Models and APIs. Hosted `gemini-flash-lite-latest` and sibling Gemini models on the free tier for selection and reading; a locally served Qwen3 through an OpenAI-compatible `llama.cpp` endpoint for text-only selection experiments, added specifically because the hosted free-tier quota (500 requests/day) is too small to score one development split twice; a cross-encoder reranker and a sentence embedding model for retrieval. The audit loop described in Section 3 was carried out by an LLM agent (Anthropic Claude) reading page text and rasterised pages, with all adjudications recorded against a specific page of a specific PDF.

Software and compute. Python, PyMuPDF for PDF text/caption/raster extraction, the organisers’ released `evaluate.py` and `validate_submission.py` for local checks. All work ran on a single workstation with no GPU training; we fine-tuned nothing.

Reproducibility. We release the pipeline, the four automated verifiers, the experiment scripts, and the reports recording each measurement including the negative ones. Prediction files and logs are not committed; the reports are the record.

10 Limitations

The most important limitation is the nature of our final submission. Our best *fully automated* run — pipeline output with no per-question intervention — scores **0.5519** (row two of Table 2); the reported 0.7647 additionally includes per-question auditing against source PDFs and corrections attributed using leaderboard feedback. The audit is not a model that would run on new questions, and the attribution is adaptation to this particular held-out split. A reader interested in what an automated system achieves on LitTraceQA should read the 0.5519 row, not the last one. We report both prominently for exactly this reason.

Second, our conventions in Section 4 are counts over 55 development questions, and several rest on small denominators (3 of 3 for the table conditional, 8 of 8 for the row-count rule). They were predictive on test, which is evidence, but they are not established at any conventional level of confidence.

Third, our weakest components are the table ones, and our analysis says why we could not fix them: matching a row key that an annotator invented is not a reasoning problem with a findable answer. This is arguably a property of the task’s table format rather than of our system, and it interacts badly with the metric — because cell accuracy is gated on row-key recall, a question whose key wording we cannot reproduce contributes zero to two of the five reported metrics no matter how correct the extracted values are.

Fourth, we did not submit to the optional `littraceqa-test-extra` diagnostic track, so we cannot report how these findings transfer to its larger question pool.

11 Ethics

Papers were retrieved from publisher-provided URLs in the released metadata and used only for evidence extraction. The dataset is CC BY-NC 4.0 and we used it within those terms. Our system’s purpose is to make claims about scientific literature checkable by pointing at a specific page of a specific paper, which we consider a beneficial direction; the corresponding risk is that a confidently formatted citation is more persuasive than an uncited claim even when wrong, and our own error analysis shows the dominant failure mode is a citation that is almost right — correct paper, adjacent row. Systems of this kind should surface the located span for human confirmation rather than presenting a grounded answer as settled.

12 Conclusion

Our LitTraceQA submission reached an official 0.7647, and the useful part of the story is where the gain came from: two-thirds of the metric is provenance rather than answering, gold’s annotation conventions are recoverable from 55 examples, and a macro metric over 71 questions is a precise enough instrument that a controlled submission can prove an individual prediction wrong. We also report the heuristics this process refuted, two bugs in our own verifiers, and the extent to which our final number reflects auditing rather than a system. For future participants we would summarise the task in one sentence: locate the evidence the way the annotator located it, and check the row and column headers before believing a value you have found.

Acknowledgements

We thank the GroundLM 2026 organisers for the released evaluator and validator, which made the analysis in this paper possible.

References

LitTraceQA dataset (CC BY-NC 4.0).

<https://huggingface.co/datasets/LitTraceQA/LitTraceQA>

GroundLM 2026 Workshop, EMNLP 2026 — shared task description, evaluator and submission rules.

https://groundlm.github.io/grouplm_emnlp2026/shared-tasks.html

GroundLM 2026 online evaluator (source of record for all held-out scores reported here).

<https://huggingface.co/spaces/YimuWang/GroundLM-2026-Eval>

PyMuPDF — Python bindings for MuPDF, used for page text, caption indices and page rasterisation.

<https://pymupdf.readthedocs.io>

ACL style files and ACL PubCheck.

<https://github.com/acl-org/acl-style-files>